

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice

Week 1: Fundamentals of Deep Learning for Generative AI

SGD update rule

where

$$W_{new} = W_{old} - \eta g$$

$$g = \frac{\partial L}{\partial W}$$

If the gradient is positive, the weight decreases.

If the gradient is negative, the weight increases.

If the gradient magnitude is large, the update is large.

If the gradient magnitude is small, the update is small.

So,:

Gradient sign decides direction, and gradient magnitude decides how much the weight changes.

Case 1: Gradient is positive

Suppose:

$$W_{old} = 5, \eta = 0.1, g = 2$$

$$W_{new} = 5 - 0.1(2)$$

$$W_{new} = 5 - 0.2 = 4.8$$

So when the gradient is **positive**, the weight **decreases**.

Case 2: Gradient is negative

Suppose:

$$W_{old} = 5, \eta = 0.1, g = -2$$

$$W_{new} = 5 - 0.1(-2)$$

$$W_{new} = 5 + 0.2 = 5.2$$

So when the gradient is **negative**, the weight **increases**.

Case 3: Gradient is large

Suppose:

$$g = 10$$

$$W_{new} = 5 - 0.1(10)$$

$$W_{new} = 5 - 1 = 4$$

The weight changed from 5 to 4.

The update is large (weight value changes by a large amount).

Case 4: Gradient is small

Suppose:

$$g = 0.5$$

$$W_{new} = 5 - 0.1(0.5)$$

$$W_{new} = 5 - 0.05 = 4.95$$

Here the weight changed from 5 to 4.95.

The update is small (weight value changes by a small amount).

Let: $W_{old} = 5$ and Large gradient: $g = 10$

Case 1: Small learning rate

Let:

$$\begin{aligned}\eta &= 0.01 \\ W_{new} &= 5 - 0.01(10) \\ W_{new} &= 5 - 0.1 \\ W_{new} &= 4.9\end{aligned}$$

Here, the weight changed from 5 to 4.9.
Small change in the weight.

Case 2: Moderate learning rate

Let:

$$\begin{aligned}\eta &= 0.1 \\ W_{new} &= 5 - 0.1(10) \\ W_{new} &= 5 - 1 \\ W_{new} &= 4\end{aligned}$$

Here, the weight changed from 5 to 4.

Case 3: Large learning rate

Let:

$$\begin{aligned}\eta &= 0.5 \\ W_{new} &= 5 - 0.5(10) \\ W_{new} &= 5 - 5 \\ W_{new} &= 0\end{aligned}$$

The weight changed from 5 to 0.

For the same gradient, learning rate controls how much the weight actually changes.

Let: $W_{old} = 5$ and small gradient: $g = 1$

Case 1: Small learning rate

Let:

$$\begin{aligned}\eta &= 0.01 \\ W_{new} &= 5 - 0.01(1) \\ W_{new} &= 5 - 0.01 \\ W_{new} &= 4.99\end{aligned}$$

The weight changed from 5 to 4.99

Case 2: Moderate learning rate

Let:

$$\begin{aligned}\eta &= 0.1 \\ W_{new} &= 5 - 0.1(1) \\ W_{new} &= 5 - 0.1 \\ W_{new} &= 4.9\end{aligned}$$

The weight changed from 5 to 4.9

Case 3: Larger learning rate

Let:

$$\begin{aligned}\eta &= 1 \\ W_{new} &= 5 - 1(1) \\ W_{new} &= 5 - 1 \\ W_{new} &= 4\end{aligned}$$

The weight changed from 5 to 4

In a real neural network, there are thousands or millions of weights. All weights may not receive gradients of the same magnitude.

Some weights may receive large gradients frequently, while some weights may receive small or rare gradients.

But standard SGD uses the same learning rate for every weight. This may not be ideal.

If a parameter is already receiving large gradient and if the learning rate is high

Let:

$$\begin{aligned}\eta &= 0.5 \\ W_{new} &= 5 - 0.5(10) \\ W_{new} &= 5 - 5 \\ W_{new} &= 0\end{aligned}$$

The weight changed from 5 to 0.
Change is too aggressive.

∴ Learning rate should be small in this scenario

If a parameter is already receiving small gradients and if the learning rate is low

Let:

$$\begin{aligned}\eta &= 0.01 \\ W_{new} &= 5 - 0.01(1) \\ W_{new} &= 5 - 0.01 \\ W_{new} &= 4.99\end{aligned}$$

The weight changed from 5 to 4.99 This is too slow.
Convergence is too slow.

∴ Learning rate should be large in this scenario

If a parameter has received large gradients many times in the past, reduce its learning rate.

If a parameter has received small or rare gradients, allow it to have a relatively larger learning rate.

So AdaGrad automatically adjusts the learning rate separately for each weight.

AdaGrad

SGD:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

If the gradient is large, the update is large
If the gradient is small, the update is small
But the same learning rate η is used for all weights

$$W_{new} = W_{old} - \eta'_t \frac{\partial L}{\partial W}$$

is an adaptive learning rate

AdaGrad changes the learning rate separately for each weight.

AdaGrad keeps track of the sum of squares of past gradients.

For one parameter:

$$\alpha_{t-1} = \sum_{i=1}^{t-1} g_i^2$$
$$g_i = \frac{\partial L}{\partial w_i}$$

This means:

- at each step, take the gradient
- square it
- add it to the previous total

So α_{t-1} stores the accumulated squared gradients from previous steps.

Then compute the new learning rate

AdaGrad uses:

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_{t-1} + \epsilon}}$$

Where,

η = initial learning rate

ϵ = very small positive number to avoid division by zero

Final AdaGrad update rule

Substitute the adaptive learning rate into the weight update:

$$w_{new} = w_{old} - \frac{\eta}{\sqrt{\alpha_{t-1} + \epsilon}} g_t$$

Since

$$\alpha_{t-1} = \sum_{i=1}^{t-1} g_i^2$$

and all squared terms are non-negative,
 α_{t-1} keeps increasing with time.

If a parameter is receiving small gradients or rare gradients, its accumulated squared gradient will remain small.

If the accumulated squared gradient α_{t-1} is small, then the denominator is small.

So:

η'_t
becomes relatively large.

That means AdaGrad allows a **larger learning rate** for parameters with small or rare gradients.

$\therefore$

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_{t-1} + \epsilon}}$$

keeps decreasing with time.

Eventually, the learning rate can become **too small**.
Then the update becomes almost zero:

$$w_{new} \approx w_{old}$$

So learning becomes extremely slow or may almost stop.

RMSProp

Unlike AdaGrad continuously growing sum, RMSProp does not accumulate all past squared gradients indefinitely.

Instead, keep only a **running average** of recent squared gradients.

RMSProp uses an **Exponentially Weighted Moving Average (EWMA)** of recent squared gradients.

This prevents the denominator from becoming too large.

At step t , compute the gradient:

$$g_t = \frac{\partial L}{\partial w_t}$$

Maintain moving average of squared gradients

Instead of

$$\text{AdaGrad: } \alpha_t = \sum_{i=1}^t g_i^2$$

RMSProp uses:

$$\text{RMSProp: } s_t = \beta s_{t-1} + (1 - \beta) g_t^2$$

where:

- s_t = exponentially weighted moving average of squared gradients at step t
- β = decay factor, usually close to 1, such as 0.9
- g_t^2 = square of the current gradient

Now RMSProp defines the effective learning rate as:

$$\eta'_t = \frac{\eta}{\sqrt{s_t} + \epsilon} \quad w_{\text{new}} = w_{\text{old}} - \frac{\eta}{\sqrt{s_t} + \epsilon} g_t$$

If: $s_t = \beta s_{t-1} + (1 - \beta) g_t^2$

then: $\beta = 0.9$

$$s_t = 0.9 s_{t-1} + 0.1 g_t^2$$

So the previous value is retained with weight 0.9, and the current squared gradient contributes with weight 0.1. Older gradients are repeatedly multiplied by 0.9, so their influence gradually decreases.

RMSProp uses:

$$s_t = \beta s_{t-1} + (1 - \beta)g_t^2$$

Let:

$$\beta = 0.9$$

Then:

$$1 - \beta = 0.1$$

So the equation becomes:

$$s_t = 0.9s_{t-1} + 0.1g_t^2$$

At step 1:

$$s_1 = 0.9s_0 + 0.1g_1^2$$

Assume:

$$s_0 = 0$$

Then:

$$s_1 = 0.9(0) + 0.1g_1^2$$

$$s_1 = \underline{0.1g_1^2}$$

At step 2:

$$s_2 = 0.9s_1 + 0.1g_2^2$$

Substitute:

$$s_1 = 0.1g_1^2$$

$$s_2 = 0.9(0.1g_1^2) + 0.1g_2^2$$

$$s_2 = \underline{0.09g_1^2} + \underline{0.1g_2^2}$$

So in s_2 , the first gradient has weight 0.09, and the second gradient has weight 0.1.

At step 3:

$$s_3 = 0.9s_2 + 0.1g_3^2$$

Substitute:

$$s_2 = 0.09g_1^2 + 0.1g_2^2$$

$$s_3 = 0.9(0.09g_1^2 + 0.1g_2^2) + 0.1g_3^2$$

$$s_3 = \underline{0.081g_1^2} + \underline{0.09g_2^2} + \underline{0.1g_3^2}$$

So the older gradient g_1^2 now has less influence than g_2^2 and g_3^2 .

At step 4:

$$s_4 = 0.9s_3 + 0.1g_4^2$$

Substitute:

$$s_3 = 0.081g_1^2 + 0.09g_2^2 + 0.1g_3^2$$

$$s_4 = 0.9(0.081g_1^2 + 0.09g_2^2 + 0.1g_3^2) + 0.1g_4^2$$

$$s_4 = \underline{0.0729g_1^2} + \underline{0.081g_2^2} + \underline{0.09g_3^2} + \underline{0.1g_4^2}$$

Now observe the weights:

$$g_1^2 \rightarrow 0.0729$$

$$g_2^2 \rightarrow 0.081$$

$$g_3^2 \rightarrow 0.09$$

$$g_4^2 \rightarrow 0.1$$

The most recent gradient has the highest weight, and older gradients have smaller weights.

ADAM

Adam = Momentum + RMSProp

Adam uses:

- **Momentum idea** → maintain EWMA of gradients
- **RMSProp idea** → maintain EWMA of squared gradients

So Adam keeps two running quantities:

1) Momentum term

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

where:

- v_t = EWMA of gradients
- $g_t = \frac{\partial L}{\partial w_t}$ = current gradient
- β_1 = momentum decay factor, usually close to 1, such as 0.9

2) RMSProp term

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

where:

- s_t = EWMA of squared gradients
- β_2 = decay factor for squared gradients, usually close to 1, such as 0.999

Use both to update the weight

Adam uses:

$$w_{new} = w_{old} - \eta \frac{v_t}{\sqrt{s_t} + \epsilon}$$

Next session

Convolutional Neural Networks: Basic Architecture Relevant to Generative Models

NPTREL

NIPTELE

Thank you